

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE CIENCIAS, 2027-I
FUNDAMENTOS DE BASES DE DATOS



PRÁCTICA 01:
Bitácora Contreras

PROFESOR:

Gerardo Avilés Rosas

AYUDANTES DE LABORATORIO:

Ricardo Badillo Macías

Rocío Aylin Huerta González

AYUDANTES DE TEORÍA:

Oscar Daniel Flores Linares

Claudia Itzel Gutiérrez Sánchez

EQUIPO:

Dobles Comillas

Información del sistema (contenerizado)

Sistema operativo y versión.

OS: Debian GNU/Linux 13 (trixie) x86_64
Kernel: Linux 7.1.13-100.fc43.x86_64
CPU: Intel(R) Core(TM) Ultra 7 155H (22) @ 3.80 GHz
Memoria: 30.70 GiB
Arquitectura: 64-bit
Contenedor: Incus *system container* con Debian 13 (trixie)

Distribución.

Distribución: Debian 13 (trixie) dentro de contenedor Incus

Versión de la instalación.

Docker version 29.8.0, build 88096ef
PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu

Tiempo requerido.

15 minutos.

Introducción: ¿Por qué Incus y por qué Debian?

Incus¹ es un gestor de contenedores de sistema y máquinas virtuales, *fork* de LXD, que permite ejecutar contenedores aislados con su propio espacio de usuario, sistema de archivos y red. Lo usamos para **aislar Docker de nuestro sistema host**: Docker requiere privilegios elevados y modifica reglas de *firewall*, redes y sistemas de archivos; ejecutarlo dentro de un contenedor Incus evita contaminar el *host* y permite desechar el entorno fácilmente.

Aunque estamos ejecutando una «doble contenerización» (Docker dentro de un contenedor Incus), esto es aceptable en este caso porque los contenedores de Incus son notablemente ligeros: comparten el *kernel* del *host*, no emulan *hardware* y usan *namespaces* y *cgroups* para aislamiento. El impacto en el rendimiento es insignificante para este caso. Al finalizar el curso, basta con ejecutar `incus rm -f debian` para eliminar todo el entorno sin dejar rastro.

Cabe señalar que el sistema base (donde se ejecuta Incus) es una distribución de Fedora, sin embargo escogimos usar debian en el contenedor pues la guía de instalación de Docker está explícitamente diseñada para sistemas basados en debian y que tienen disponible el gestor de paquetes apt. De esta forma no fue necesario hacer ninguna adaptación al procedimiento estandar para instalar Docker.

Procedimiento

i. Crear contenedor basado en Debian mediante Incus.

¹Proyecto oficial: <https://linuxcontainers.org/incus/> (también en <https://zabbly.com/incus/>).

En el *host*, creamos un contenedor de sistema Incus basado en una imagen de Debian 13 (trixie):

```
incus launch images:debian/13/trixie debian --profile default
```

A partir de aquí entramos a una *shell* dentro del contenedor debian usando:

```
incus shell debian
```

Cabe señalar que el único usuario que existe por defecto dentro del contenedor es *root*. Por ahora esto no presenta problemas pues de todas formas todos los pasos de instalación de Docker y posteriormente el contenedor con Postgres requieren privilegios de superusuario. Además, la protección real viene de que el contenedor de Incus se mantiene aislado del sistema anfitrión.

Dentro del contenedor, verificamos el sistema:

```
root@debian ~# fastfetch
_,met$$$$$g.
,g$$$$$$$$$SP.
,g$P"" ""Y$.
,$$P' '$$.
',$$P',ggs. '$b:
`d$$' ,SP"'. $$$
$$P d$' , $$P
$$: $$: ,d$$'
$$; Y$b._ _dSP'
Y$$. `."Y$$$$P"
`$$b "-._
`Y$b
`Y$.
`$$b.
`Y$b.
`"Y$b._
`""
root@debian ~#
```

```
root@debian
-----
OS: Debian GNU/Linux 13 (trixie) x86_64
Host: ASUS Zenbook 14 UX3405MA_UX3405MA (1.0)
Kernel: Linux 7.1.13-100.fc43.x86_64
Uptime: 37 mins
Packages: 253 (dpkg)
Shell: fish 4.0.2
Display (SDC419D): 2880x1800 @ 60 Hz in 14" [Built-in]
Display (LG HDR 4K): 3840x2160 @ 60 Hz in 31" [External]
Terminal: foot 1.25.0
Terminal Font: monospace (8pt)
CPU: Intel(R) Core(TM) Ultra 7 155H (22) @ 3.80 GHz
GPU: Intel Device 7D55 (VGA compatible) @ 2.25 GHz [Integrated]
Memory: 122.41 MiB / 30.70 GiB (0%)
Swap: 0 B / 8.00 GiB (0%)
Disk (/): 522.14 GiB / 820.94 GiB (64%) - btrfs
Local IP (eth0): 10.255.182.198/24
Battery (ASUS Battery): 60% [AC Connected]
Locale: en_US.UTF-8
```

ii. Actualizar lista de paquetes.

```
apt update
```

```
root@debian ~# apt update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
All packages are up to date.
root@debian ~#
```

iii. Instalar dependencias para agregar repositorio Docker. (ya estaban instaladas)

```
apt install -y ca-certificates curl
```

```
root@debian ~# apt install ca-certificates curl
ca-certificates is already the newest version (20250419).
curl is already the newest version (8.14.1-2+deb13u4).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading
: 0
root@debian ~#
```

iv. **Crear directorio para *keyrings*.**

```
install -m 0755 -d /etc/apt/keyrings
```

```
root@debian ~# install -m 0755 -d /etc/apt/keyrings
root@debian ~#
```

v. **Descargar llave GPG oficial de Docker.**

```
curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/
docker.asc
```

```
root@debian ~# curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
root@debian ~#
```

vi. **Dar permisos de lectura a la llave.**

```
chmod a+r /etc/apt/keyrings/docker.asc
```

```
root@debian ~# chmod a+r /etc/apt/keyrings/docker.asc
root@debian ~#
```

vii. **Agregar repositorio oficial de Docker.**

```
tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

```
root@debian:~# tee /etc/apt/sources.list.d/docker.sources <<EOF
> Types: deb
> URIs: https://download.docker.com/linux/debian
> Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
> Components: stable
> Architectures: $(dpkg --print-architecture)
> Signed-By: /etc/apt/keyrings/docker.asc
> EOF
```

```
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: trixie
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
root@debian:~#
```

viii. **Actualizar índice de paquetes con el nuevo repositorio.**

```
apt update
```

```
root@debian ~# apt update
Get:1 https://download.docker.com/linux/debian trixie InRelease [32.5 kB]
Hit:2 http://deb.debian.org/debian trixie InRelease
Get:3 https://download.docker.com/linux/debian trixie/stable amd64 Packages [45.3 kB]
Hit:4 http://deb.debian.org/debian trixie-updates InRelease
Hit:5 http://deb.debian.org/debian-security trixie-security InRelease
Fetched 77.8 kB in 0s (264 kB/s)
All packages are up to date.
root@debian ~#
```

ix. Instalar Docker Engine y plugins.

`apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`

```
root@debian ~# apt install docker-ce docker-ce-cli containerd.io d
ocker-buildx-plugin docker-compose-plugin
```

```
sbin/ebtables (ebtables) in auto mode
Setting up docker-ce (5:29.8.0-1~debian.13~trixie) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' → '/usr/lib/systemd/system/docker.service'.
Created symlink '/etc/systemd/system/sockets.target.wants/docker.socket' → '/usr/lib/systemd/system/docker.socket'.
Processing triggers for libc-bin (2.41-12+deb13u3) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for mailcap (3.74) ...
root@debian ~#
```

x. Verificar que el servicio Docker está activo.

`systemctl status docker`

```

^● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Drop-In: /run/systemd/system/service.d
            └─zzz-lxc-service.conf
   Active: active (running) since Sun 2026-09-06 00:57:28 UTC; 32s ago
   Invocation: 412976341bc24e28a96619272a21125d
   TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 2381 (dockerd)
   Tasks: 17
   Memory: 28.3M (peak: 30.7M)
   CPU: 687ms
   CGroup: /system.slice/docker.service
           └─2381 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Sep 06 00:57:27 debian dockerd[2381]: time="2026-09-06T00:57:27.248497392Z" level=info ms>
Sep 06 00:57:27 debian dockerd[2381]: time="2026-09-06T00:57:27.275288292Z" level=info ms>
Sep 06 00:57:27 debian dockerd[2381]: time="2026-09-06T00:57:27.297751106Z" level=info ms>
Sep 06 00:57:27 debian dockerd[2381]: time="2026-09-06T00:57:27.838049041Z" level=info ms>
Sep 06 00:57:27 debian dockerd[2381]: time="2026-09-06T00:57:27.850307680Z" level=info ms>
Sep 06 00:57:27 debian dockerd[2381]: time="2026-09-06T00:57:27.850409160Z" level=info ms>
Sep 06 00:57:28 debian dockerd[2381]: time="2026-09-06T00:57:28.032782083Z" level=info ms>
Sep 06 00:57:28 debian dockerd[2381]: time="2026-09-06T00:57:28.041918094Z" level=info ms>
Sep 06 00:57:28 debian dockerd[2381]: time="2026-09-06T00:57:28.042020318Z" level=info ms>
Sep 06 00:57:28 debian systemd[1]: Started docker.service - Docker Application Container >
~
~
lines 1-25/25 (END)

```

xi. Probar Docker con la imagen hello-world.

```
docker run hello-world
```

```

root@debian ~# docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

root@debian ~#

```

xii. Descargar imagen oficial de PostgreSQL.

```
docker pull postgres
```

```

root@debian ~# docker pull postgres
Using default tag: latest
latest: Pulling from library/postgres
2fc0e69f8d7d: Pull complete
5fa29c524b4e: Pull complete
6310eb16bf42: Pull complete
8d6422da287d: Pull complete
c5694bff1357: Pull complete
af2a6480a058: Pull complete
aedab7177504: Pull complete
ace4b5b96a91: Pull complete
46e663abc8f5: Pull complete
5ee6c6ca7fd9: Pull complete
1bb54e8f7727: Pull complete
6463c6b5c703: Pull complete
a4635eaec9b8: Pull complete
041cb76c0e1d: Download complete
6ff88415323f: Download complete
Digest: sha256:4ef4dbc939d61acea57712655ddb4b4ab27419c913f94cca0cd57cb3ea3c2280
Status: Downloaded newer image for postgres:latest
docker.io/library/postgres:latest
root@debian ~# █

```

xiii. Verificar imágenes descargadas.

`docker images`

```

root@debian ~# docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
53004db15d41   postgres  "docker-entrypoint.s..." About a minute ago Up About a minute      127.0.0.1:5432->5432/tcp  postgres
674a5aed496d   hello-world  "/hello"                6 minutes ago Exited (0) 6 minutes ago          festive_keller
da84baa88880   hello-world  "/hello"                6 minutes ago Exited (0) 6 minutes ago          elated_gould
root@debian ~# █

```

xiv. Crear y ejecutar contenedor PostgreSQL.

```

docker run -d \
  --name postgres \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -p 127.0.0.1:5432:5432 \
  -v postgres_data:/var/lib/postgresql/data \
  postgres

```

```

root@debian ~# docker run -d \
  --replace \
  --name postgres \
  -e POSTGRES_PASSWORD=doblescomillas \
  -p 0.0.0.0:5432:5432 \
  -v postgres_data:/var/lib/postgresql \
  postgres

```

xv. Obtener la IP del contenedor Incus para conexiones externas.

El contenedor PostgreSQL expone el puerto 5432 en la interfaz de red del contenedor Incus (no en *localhost* del *host*). Para conocer la *IP*, desde el *host* ejecutamos:

```
sudo incus list
```

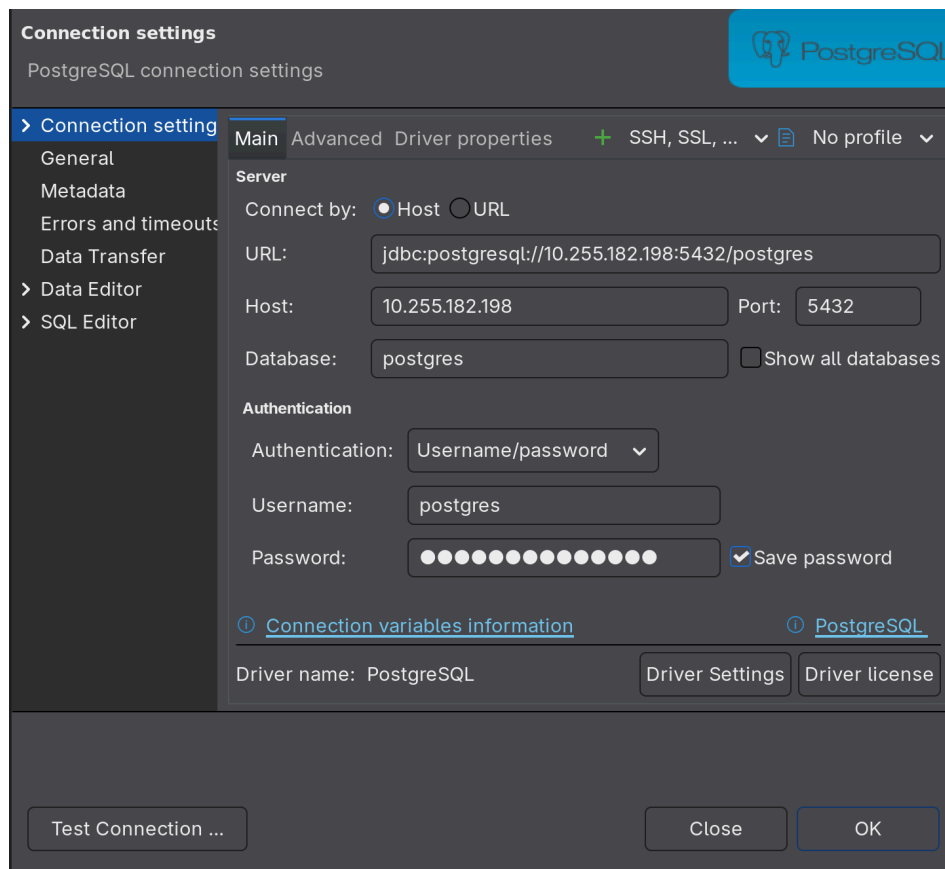
La salida muestra la *IPv4* del contenedor (ej. 10.255.182.198). Esa es la dirección a usar en DBeaver como *Host*.

```
roman@zenbook: ~$ sudo incus list
```

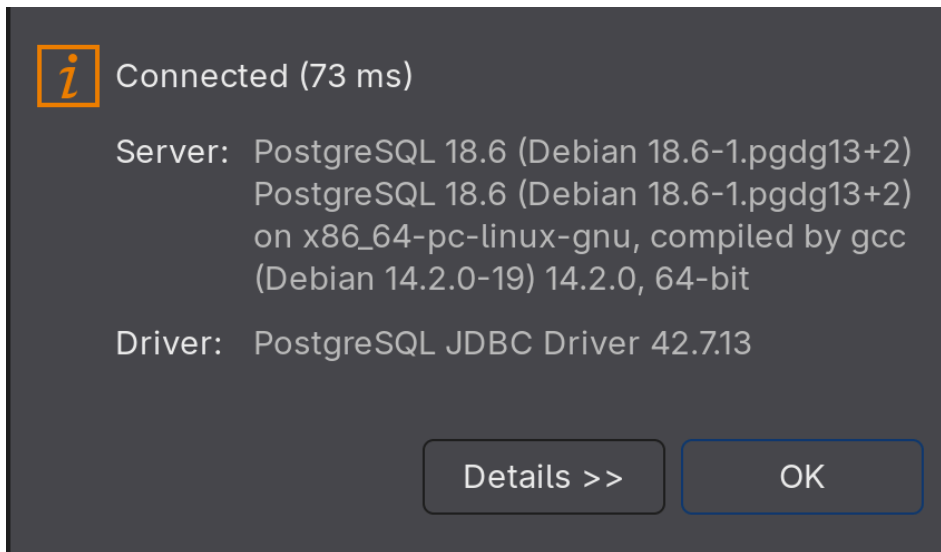
NAME	STATE	IPV4	IPV6	TYPE	SNAPSHOTS
debian	RUNNING	172.17.0.1 (docker0)	fd42:16be:3c7:4d8:1266:6aff:fe71:149 (eth0)	CONTAINER	0
		10.255.182.198 (eth0)			

xvi. Configurar DBeaver para conectar al contenedor.

En DBeaver, nueva conexión → PostgreSQL → Host: 10.255.182.198 (IP del contenedor Incus), Puerto: 5432, Base de datos: postgres, Usuario: postgres, Contraseña: mysecretpassword. *Driver* descargado automáticamente.



xvii. Probar conexión



Comentarios y problemas

El uso de Incus añadió un paso inicial (crear el contenedor), pero simplificó todo lo demás: el entorno Debian está recién creado, por lo que está limpio y sin conflictos de paquetes del *host*. La red del contenedor expone el puerto 5432 en su propia *IP*. La guía oficial de Docker para Debian funcionó sin modificaciones.